

Planax: A JAX-Accelerated High-Fidelity Reinforcement Learning Benchmark for Large-Scale Fixed-Wing Aerial Swarms

Anonymous Authors

Abstract—Multi-agent reinforcement learning (MARL) for fixed-wing aerial robots currently faces a critical bottleneck: the severe trade-off between physical realism and training throughput. Existing open-source platforms often rely on simplified point-mass kinematics, failing to capture the non-holonomic constraints and energy maneuverability of highly dynamic flights. Conversely, traditional high-fidelity aviation simulators suffer from severe computational bottlenecks, rendering large-scale parallel MARL infeasible. To address this, we propose Planax, a JAX-accelerated, fully GPU-resident high-fidelity simulation framework. By leveraging XLA compilation and operator fusion, Planax integrates a nonlinear six-degree-of-freedom (6-DOF) flight dynamics model—driven by authentic aerodynamic wind-tunnel data via branch-free tensor operations—directly into optimized GPU kernels. Comprehensive experiments demonstrate that Planax achieves a substantially higher simulation throughput exceeding 2.3×10^7 steps-per-second on a single NVIDIA A100 GPU and supports up to 2×10^7 concurrent 6-DOF agents (OOM limit: 4.7×10^7). The framework successfully stabilizes massive decentralized formation keeping and hierarchical pursuit-evasion benchmarks. Crucially, a rigorous sim-to-real proxy ablation reveals that policies trained without our high-fidelity constraints exploit physical loopholes, suffering a 100% crash rate due to structural overloads during agile maneuvers. In contrast, policies trained in Planax exhibit substantially improved survival and physically grounded maneuvering under the tested high-fidelity constraints. The Planax codebase and video demonstrations are open-source and available at: <https://anonymous.4open.science/r/Aeroplanax-2087>.

Index Terms—Aerial Systems; Mechanics and Control, Multi-Agent Reinforcement Learning, Swarm Robotics, GPU-Accelerated Simulation, High-Fidelity Benchmarking.

I. INTRODUCTION

THE deployment of large-scale multi-agent reinforcement learning (MARL) for autonomous aerial systems is rapidly transitioning from conceptual exploration to physical implementation. While MARL has demonstrated remarkable success in coordinating complex multi-robot behaviors, its application to high-dynamic fixed-wing aerial robots remains severely constrained by a fundamental “sim-to-real” gap. Unlike hovering quadrotors, fixed-wing flight is dominated by strict non-holonomic constraints, complex rigid-body coupling, and severe energy maneuverability limitations. Existing MARL abstractions often simplify these vehicles into kinematic point-masses. Such simplifications allow policies to exploit non-physical loopholes during training, rendering the learned cooperative or adversarial strategies undeployable in the real world. Overcoming this gap necessitates training environments that enforce rigorous physical realism.

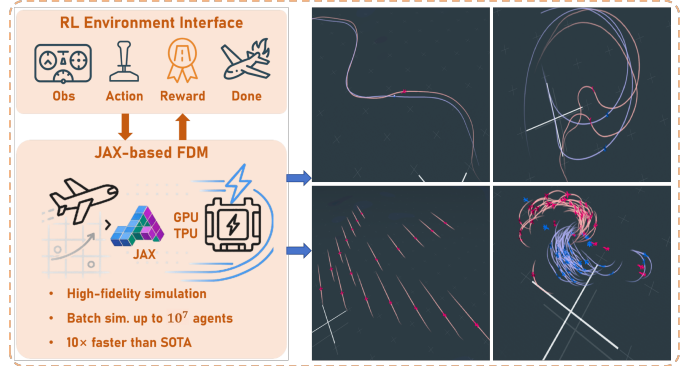


Fig. 1. **Overview of the Planax framework.** (Left) The JAX-accelerated simulation architecture and reinforcement learning (RL) environment interface. (Right) Tacview visualization of representative benchmark results, including single-agent high-dynamic S-maneuvers, 2v2 adversarial pursuit-evasion, large-scale competitive swarm interactions, and multi-agent cooperative formation flight.

However, achieving such high-fidelity realism introduces a critical computational bottleneck for MARL, which typically requires millions to billions of environmental interactions. Currently, the simulation ecosystem presents a severe dichotomy. On one hand, widely adopted robotics RL frameworks (e.g., Gym-PyBullet-Drones, Isaac Gym) excel at massively parallel, GPU-accelerated execution but are predominantly designed for ground robots or quadrotors, lacking the infrastructure for sophisticated aerodynamic modeling. On the other hand, traditional high-fidelity aviation simulators (e.g., JSBSim) provide validated six-degree-of-freedom (6-DOF) flight dynamics driven by authentic wind-tunnel data. Yet, these engines are strictly CPU-centric. The resultant host-device memory transfer overheads and fragmented sequential execution make them computationally prohibitive for large-scale parallel policy optimization.

To bridge the gap between simulation throughput and physical fidelity, we propose Planax, a massively parallel, JAX-accelerated simulation framework tailored for large-scale fixed-wing aerial swarms. As illustrated in Fig. 1, Planax eschews the conventional “CPU-simulator paired with GPU-policy” paradigm. Instead, it leverages XLA (Accelerated Linear Algebra) compilation and operator fusion to integrate a rigorous 6-DOF nonlinear flight dynamics model directly into GPU-resident SIMD kernels. This end-to-end device residence completely eliminates PCIe memory transfer latencies. Most distinctively, Planax processes multi-dimensional authentic

aerodynamic data through highly optimized, branch-free tensor operations at the XLA operator level, avoiding the physical distortion of surrogate approximations while maintaining maximum parallel efficiency.

The main contributions of this letter are threefold:

- We introduce a JAX-based, fully GPU-resident MARL benchmark framework for high-dynamic fixed-wing aerial robots, achieving a simulation throughput exceeding 2.3×10^7 steps-per-second (SPS) on a single NVIDIA A100 GPU under the evaluated configuration.
- We develop a native tensorized implementation of complex aerodynamic lookup tables (LUTs) directly within XLA-fused kernels. This design numerically matches the tabulated aerodynamic coefficients under the same interpolation rule while avoiding the additional approximation error introduced by neural aerodynamic surrogates.
- We provide an open-source benchmark suite covering single-agent agile tracking, decentralized formation keeping, and adversarial pursuit-evasion. Through controlled fidelity ablations, we show that high-fidelity aerodynamic constraints substantially affect policy safety and transfer behavior, and we validate compatibility with mainstream MARL algorithms.

II. RELATED WORK

A. Aerial Robot Simulators

The advancement of reinforcement learning (RL) for aerial robots is intrinsically linked to the evolution of simulation fidelity and throughput. Traditional high-fidelity flight simulators, most notably JSBSim [1] and X-Plane [2], provide rigorous nonlinear six-degree-of-freedom (6-DOF) dynamics driven by authentic wind-tunnel data. While these engines ensure physical credibility, their CPU-centric serial execution models create a severe computational bottleneck, making it impractical to generate the billions of interaction steps required by modern multi-agent reinforcement learning (MARL) algorithms at scale.

To overcome these limitations, the robotics community has pivoted toward GPU-accelerated and benchmark-oriented simulation platforms. Frameworks such as *Gym-PyBullet-Drones* [3], *Flightmare* [4], *OmniDrones* [5], and *Isaac Gym* [6] leverage hardware acceleration or modular simulation design to support robot learning experiments. However, these environments are predominantly optimized for hovering quadrotors or simplified point-mass kinematics. They frequently lack the specialized aerodynamic primitives—such as multi-dimensional lift/drag look-up tables (LUTs) and complex inertial coupling—necessary to capture the non-holonomic constraints and energy maneuverability characteristic of high-dynamic fixed-wing flight.

Planax occupies a unique ecological niche by unifying these paradigms. As summarized in Table I, while existing platforms compromise on either multi-agent scalability, physical fidelity, or execution efficiency, Planax satisfies all constraints. Building upon the foundational concept of parallelized flight dynamics introduced by *NeuralPlane* [7], Planax extends the capability into the multi-agent regime. By integrating native

tensorized aerodynamic operations within JAX-accelerated ecosystems [8], [9], [10], Planax reduces a key source of the sim-to-real gap without sacrificing training efficiency.

B. MARL Benchmarks for High-Dynamic Systems

Benchmark suites have played a central role in robot learning and learning-based control, as demonstrated by manipulation and safe-control benchmarks such as RL Bench [11] and safe-control-gym [12]. Standard MARL benchmarks, such as the Multi-Agent Particle Environment (MPE) [13] and the StarCraft Multi-Agent Challenge (SMAC) [14], have also been instrumental in the development of cooperative and competitive algorithms. Nevertheless, these testbeds primarily focus on low-dimensional grid-worlds or kinematic navigation, lacking continuous-time, physics-consistent aerodynamic constraints that dictate the feasibility of aerial maneuvers.

Recent aeronautics-oriented studies have begun emphasizing cooperative decision-making in more realistic settings, proposing platforms like QPlane [15] and various hierarchical adversarial pursuit-evasion environments [16], [17]. However, these tools are often limited by scalability or lack end-to-end device residence. In contrast, Planax provides a native, large-scale MARL infrastructure that supports high-dynamic pursuit-evasion and dense formation tasks under strict flight-envelope constraints, establishing a new standard for evaluating decentralized coordination in realistic aerial environments.

III. SYSTEM ARCHITECTURE

This section describes the architectural design of Planax (depicted in Fig. 2), focusing on how it reconciles high-fidelity aerodynamic modeling with the massive throughput requirements of multi-agent reinforcement learning (MARL).

A. GPU-Resident Flight Dynamics

To bridge the sim-to-real gap, Planax implements a nonlinear six-degree-of-freedom (6-DOF) flight dynamics model that strictly adheres to the physical laws of high-dynamic flight. By executing the entire dynamics chain on the GPU, we minimize the latency typically associated with heterogeneous simulation-learning loops.

1) *Aircraft State and Kinematics*: Each aircraft is represented by a 13-dimensional state vector $\mathbf{x} = [u, v, w, p, q, r, x_E, y_E, z_E, q_0, q_x, q_y, q_z]^T$, including body-frame translational velocities, body-frame angular rates, inertial position, and attitude. Following standard rigid-body flight simulation practice, attitude is represented by a unit quaternion

TABLE I
COMPARISON OF AERIAL SIMULATION PLATFORMS FOR MARL

Platform	Native MARL	GPU / JIT	Authentic Aero.	Device-Resident
X-Plane [2]	No	No	Yes (Standard LUT)	No (CPU)
JSBSim [1]	No	No	Yes (Standard LUT)	No (CPU)
Isaac Gym [6]	Yes	Yes	No (Point-mass)	Yes (Tensor API)
NeuralPlane [7]	Limited	Yes	Yes (MLP Surrogate)	Yes (Eager Tensors)
Planax (Ours)	Yes	Yes (JAX)	Yes (Tensor LUT)	Yes (XLA Fused)

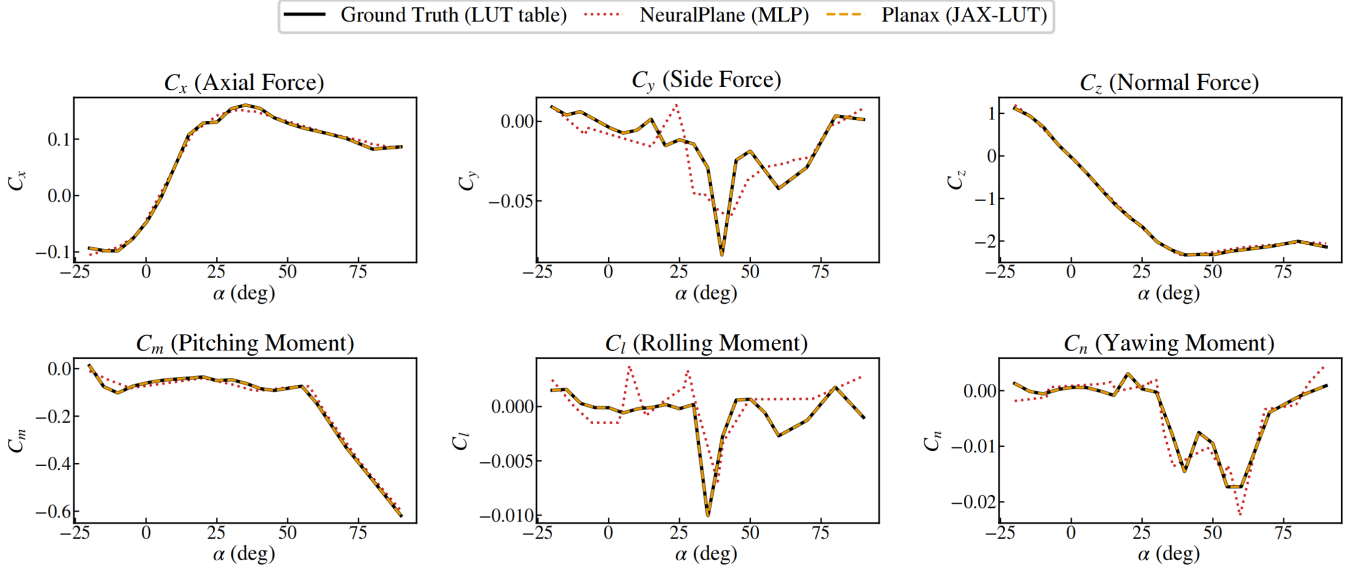


Fig. 3. **Aerodynamic Fidelity Evaluation.** Comparison of the six F-16 aerodynamic coefficients ($C_x, C_y, C_z, C_l, C_m, C_n$) computed by Planax’s branch-free JAX-LUT, NeuralPlane’s PyTorch MLP surrogate, and the Tabulated aerodynamic data across varying angles of attack α (under fixed conditions: sideslip $\beta = 0^\circ$, elevator deflection $\delta_e = 0^\circ$). While the MLP surrogate exhibits noticeable systemic bias and boundary degradation at extreme maneuvers (e.g., high α stall regions in C_z and C_m), Planax numerically matches the tabulated coefficients under the same interpolation rule, avoiding the additional approximation error introduced by the neural surrogate.

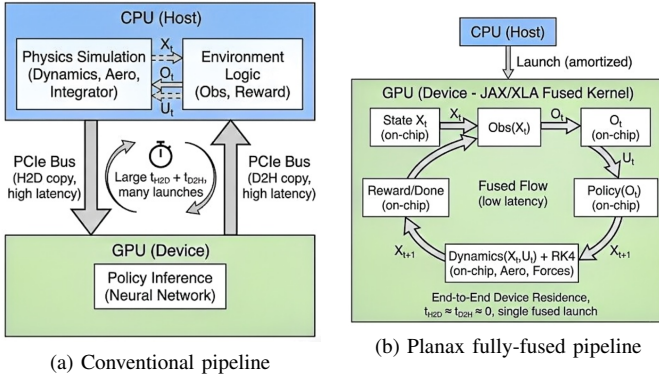


Fig. 4. **End-to-end throughput bottlenecks.** (a) The conventional MARL pipeline suffers from severe PCIe memory transfer latencies (T_{H2D}, T_{D2H}) and fragmented kernel launches. (b) Planax leverages XLA compilation to fuse the 6-DOF dynamics, aerodynamic Tensor LUTs, and reward logic into a single GPU-resident kernel, eliminating transfer overheads.

the single-agent transition function into a highly efficient, batched SIMD (Single Instruction, Multiple Data) operation. This allows the state updates for millions of agents to be executed concurrently across the GPU hardware grid without any Python-level loop overhead.

2) *XLA Operator Fusion and Loop Unrolling:* In complex physical simulations involving RK4 integration and multi-dimensional aerodynamics, sequentially executing fragmented operations leads to excessive global memory read/write latencies (T_{mem}). Planax utilizes `jax.lax.scan` to unroll the temporal simulation loop and applies Just-In-Time (`jit`) compilation powered by the XLA compiler. As illustrated in Fig. 4, XLA analyzes the entire computational graph and fuses the physics equations into optimized GPU kernels, as

Algorithm 1 End-to-End JAX Compilation Pipeline in Planax

Require: Base dynamics $f_{6DOF}(\mathbf{x}, \mathbf{u})$, tensorized aerodynamic lookup tables $\mathcal{G}_{aero}$

Ensure: Fused rollout kernel `Rolloutjit`

1: **Define single-agent step as a pure function:**

- 2: $(\alpha_t, \beta_t, \delta_t) \leftarrow \text{AeroInputs}(\mathbf{x}_t, \mathbf{u}_t)$
- 3: $\mathbf{C}_{aero} \leftarrow \text{TensorLUT}_{aero}(\mathcal{G}_{aero}, \alpha_t, \beta_t, \delta_t)$
- 4: $\dot{\mathbf{x}}_t \leftarrow f_{6DOF}(\mathbf{x}_t, \mathbf{u}_t, \mathbf{C}_{aero})$
- 5: $\mathbf{x}_{t+1} \leftarrow \text{RK4_Integrate}(f_{6DOF}, \mathbf{x}_t, \mathbf{u}_t, \mathcal{G}_{aero})$
- 6: $o_{t+1}, r_t, d_t \leftarrow \text{TaskLogic}(\mathbf{x}_{t+1})$
- 7: $\text{step}_1(\mathbf{x}_t, \mathbf{u}_t) \mapsto (\mathbf{x}_{t+1}, o_{t+1}, r_t, d_t)$

8: **Batch vectorization over the agent axis:**

- 9: $\text{Step}_{batch} \leftarrow \text{jax.vmap}(\text{step}_1, \text{in_axes} = (0, 0))$

10: **Time unrolling over the rollout horizon:**

- 11: $\text{Rollout} \leftarrow \text{jax.lax.scan}(\text{Step}_{batch}, \mathbf{X}_0, \mathbf{U}_{1:T})$

12: **XLA compilation and operator fusion:**

- 13: $\text{Rollout}_{jit} \leftarrow \text{jax.jit}(\text{Rollout})$

14: **return** `Rolloutjit` executed entirely on GPU VRAM

summarized in Algorithm 1. Unlike PyTorch-based aerial simulators (e.g., NeuralPlane) that suffer from severe Python-level dispatch overheads and fragmented global memory accesses during eager tensor execution, Planax’s compiled architecture executes the entire complex aerodynamic and kinematic chain as a monolithic device operation.

3) *Zero-Transfer Simulation-Learning Loop:* Because both the environment rollout and the MARL policy updates (e.g. PPO/MAPPO) are compiled into unified XLA computational graphs, experience replay buffers, trajectory sampling, and gradient descent all occur entirely within the GPU VRAM. This “zero-transfer” architecture eliminates the PCIe bottle-

neck, shifting the scalability limits directly to the raw compute and memory capacity of the GPU.

4) *Numerical Reliability and Safety Checks*: Large-scale parallel rollout of nonlinear 6-DOF aircraft dynamics requires explicit handling of numerical validity. In Planax, the environment step includes vectorized checks for finite states, bounded aerodynamic angles, flight-envelope violations, and out-of-bound positions. The unit quaternion is renormalized after state propagation to suppress numerical drift, following standard practice in rigid-body simulation. In addition, unsafe flight conditions, such as aerodynamic stall, excessive load factor, and collision, are encoded as vectorized Boolean masks and used to update the termination flag within the JAX/XLA-compiled environment step. These checks are compiled together with the dynamics, observation, reward, and termination logic, preserving the fully GPU-resident execution pattern without introducing host-side synchronization.

C. Task Environments and Benchmark Suite

Planax provides a unified benchmark suite for fixed-wing aerial reinforcement learning under high-fidelity 6-DOF dynamics and flight-envelope constraints. The suite covers three representative settings: single-agent agile tracking, cooperative formation keeping, and adversarial pursuit-evasion. Each environment follows the same batched interaction interface, where agents receive normalized observations, execute either discretized actuator commands or discretized high-level reference commands, and return rewards and termination flags in tensorized form. The simulation runs at 50 Hz ($\Delta t_{\text{sim}} = 0.02$ s), while policy actions are applied every 10 simulation steps, corresponding to a control interval of 0.20 s. Table II summarizes the task interfaces and evaluation metrics used in the Planax benchmark suite.

1) *Single-Agent Agile Tracking*: Single-agent tasks evaluate whether a policy can track commanded flight states under nonlinear fixed-wing dynamics. The observation contains ego flight quantities such as tracking errors, airspeed, altitude, attitude features, aerodynamic angles, and body rates. The direct-control variants use discretized actuator commands for throttle, elevator, aileron, and rudder.

2) *Cooperative Formation Keeping*: In the formation task, multiple aircraft maintain a prescribed geometric pattern while respecting flight-envelope and collision constraints. Each agent observes its own normalized flight state and relative geometry to nearby agents. The policy outputs high-level reference commands for desired pitch, heading, and airspeed, which are tracked by a low-level controller.

3) *Adversarial Pursuit-Evasion*: The adversarial pursuit-evasion task evaluates competitive multi-agent interaction under identical fixed-wing dynamics. Each agent observes its own flight state and range-limited relative geometry to other agents. The objective is to improve relative positioning while maintaining safe flight, rather than exploiting unconstrained point-mass motion.

Unsafe flight conditions, including low altitude, low speed, excessive load factor, extreme angle of attack, excessive sideslip, collision, and out-of-bound states, are treated as

episode termination conditions. Reward functions are task-specific: tracking tasks penalize attitude and speed errors, formation tasks penalize slot deviation, and pursuit-evasion tasks combine relative-geometry and event-based terms. Full task definitions, configuration files, and evaluation scripts are released with the codebase to support reproducible benchmarking.

IV. EXPERIMENTS AND BENCHMARKING

The empirical evaluation of Planax focuses on four aspects of its effectiveness as a MARL infrastructure: (1) simulation throughput and scalability compared with traditional simulators; (2) coefficient-level aerodynamic fidelity of the tensorized LUT engine; (3) the effect of aerodynamic fidelity on policy safety under agile maneuvers; and (4) the ability of the framework to support stable large-scale MARL training. All computational benchmarks and RL training are conducted on a single NVIDIA A100 (80GB VRAM) GPU and an AMD EPYC 64-core CPU. Unless otherwise stated, policies are trained using PPO-family algorithms with generalized advantage estimation. The released training configurations use $\gamma = 0.99$, GAE $\lambda = 0.95$, PPO clipping coefficient 0.20, gradient clipping, and recurrent policy networks with 128-dimensional hidden states. Evaluation is performed with greedy actions over fixed scenario settings, and results are summarized using mission-level metrics such as return, episode length, success rate, formation error, and safety-violation statistics.

A. Simulation Throughput and Scalability

To quantify the computational efficiency of Planax, we measure simulation throughput in environment steps per second (SPS) and GPU memory footprint under a matched benchmarking protocol. Throughput is reported after JIT warm-up and excludes one-time compilation overhead. For Planax, one environment step includes 6-DOF state propagation, TensorLUT aerodynamic evaluation, observation construction, reward computation, and termination checks within the compiled environment step. JSBSim is evaluated in both single-thread and multi-process CPU configurations, while NeuralPlane is evaluated using its GPU-parallel rollout setting whenever the same aircraft count and physical simulation time are supported. GPU memory is recorded after warm-up using the peak allocated VRAM during rollout.

1) *Throughput Analysis (SPS)*: As illustrated in Fig. 5 (Left), traditional CPU-centric simulators exhibit severe scaling limitations. While multi-processing provides a linear speedup for small populations, the throughput of JSBSim rapidly saturates due to context-switching overhead and Inter-Process Communication (IPC) bottlenecks, peaking at approximately 2×10^3 SPS even on a 64-core CPU.

Conversely, Planax leverages `vmap` vectorization to achieve massive SIMD parallelism. For small agent populations ($N < 10^4$), the throughput is primarily bounded by the XLA kernel launch overhead rather than compute latency, remaining near-constant at approximately 7 ms per step. As the population grows, the GPU compute cores become progressively saturated, reaching a peak throughput of approximately 2.3×10^7

TABLE II
SUMMARY OF PLANAX BENCHMARK TASKS AND INTERFACES.

Task	Agents	Observation	Action	Main metrics
Single-agent agile tracking	1	16–22D ego tracking state	4-channel discretized actuator command	Tracking error, return, episode length, safety violations
Cooperative formation keeping	Variable N	Ego state and relative neighbor geometry	3-channel discretized high-level command	Formation error, success rate, return, safety violations
Adversarial pursuit-evasion	Team-based	Ego state and range-limited relative geometry	3-channel discretized high-level command	Success rate, relative-position score, return, episode length

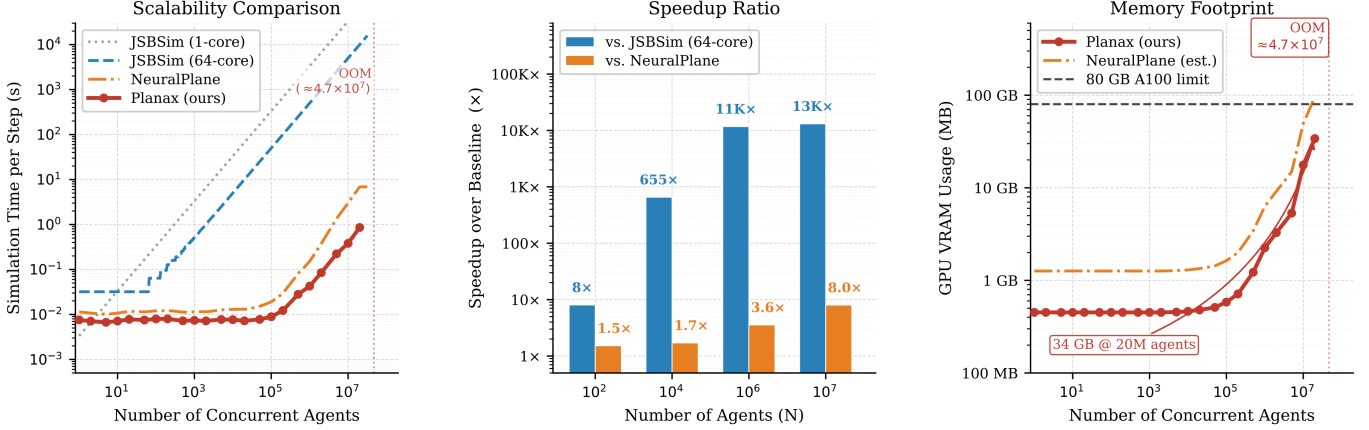


Fig. 5. **Scalability and Throughput Analysis.** (Left) Simulation time required versus the number of concurrent agents (log-log scale). Planax entirely avoids the exponential time growth seen in the CPU-centric JSBSim. (Middle) Speed-up ratio benchmark. At massive scales (e.g., 10^7 agents), Planax achieves a $8\times$ acceleration over the PyTorch-based NeuralPlane, validating the efficiency of our XLA-fused, branch-free architecture. (Right) GPU memory footprint (VRAM). Planax exhibits a uniquely flat initial scaling curve dominated by the static compiled graph footprint. Its memory efficiency supports 2×10^7 concurrent high-fidelity agents consuming only 34GB VRAM on a single A100 80GB GPU, with the extrapolated OOM limit at $\sim 4.7 \times 10^7$ agents.

SPS at $N \approx 2 \times 10^6$ agents. Beyond this compute-saturated regime, throughput transitions into a memory-bandwidth-bound regime where simulation time scales linearly with N . The platform operates successfully up to $N \approx 2 \times 10^7$ agents (34 GB VRAM), with the estimated single-GPU out-of-memory (OOM) limit at $N \approx 4.7 \times 10^7$. This peak throughput represents an improvement of over four orders of magnitude ($> 10,000\times$) over the 64-core JSBSim baseline, fundamentally enabling the generation of billions of transition samples—required for stabilizing complex MARL—within hours instead of weeks.

2) *VRAM Memory Footprint*: Massive parallelism is only practical if the memory footprint remains tractable. Fig. 5 (Right) demonstrates the GPU VRAM consumption of Planax as a function of the agent population. Because the XLA compiler heavily fuses operations and eliminates the need to allocate intermediate tensors (e.g., instantaneous aerodynamic forces and RK4 stages) in global memory, the per-agent memory cost is drastically minimized.

The memory growth curve remains remarkably flat for $N < 10^5$. This is because the baseline VRAM is dominated by the static footprint of the compiled XLA computational graph and the tensorized F-16 aerodynamic parameters. Linear scaling is observed only at extreme population scales, allowing Planax to comfortably support up to 2×10^7 concurrent 6-DOF agents on a single 80GB A100 GPU (occupying only 34 GB), with the estimated Out-Of-Memory (OOM) limit at approximately 4.7×10^7 agents.

B. Multi-Agent Benchmark Validation

Preliminary evaluations confirmed that individual agents readily converge on agile single-control tasks (e.g., heading tracking) under the F-16 6-DOF dynamics, laying the foundation for complex multi-agent interactions. Before analyzing the critical impact of physical fidelity (Section IV-C), we first validate that Planax provides a mathematically stable and efficient optimization landscape for standard MARL algorithms. We benchmarked the platform using Independent PPO (IPPO) [18] for decentralized cooperative tasks and a Hierarchical MAPPO (HRL) [19] architecture for adversarial tasks.

1) *Cooperative Formation Keeping*: Fig. 6 (Top) illustrates the training returns of the IPPO algorithm across varying swarm sizes and geometric templates (e.g., wedge, line, diamond). Despite the highly coupled 6-DOF aerodynamics and strict collision avoidance constraints, the decentralized policies demonstrate stable and monotonic convergence.

Notably, Planax successfully trains massive swarms (up to $N = 50$) within 3×10^8 environmental steps. Additional cross-scale evaluation scripts are released with the codebase for reproducibility. This confirms that the decentralized observation interface in Planax successfully encapsulates scalable, scale-invariant spatial coordination rules.

2) *Adversarial Pursuit-Evasion and Hierarchy*: In the highly non-stationary adversarial pursuit-evasion benchmark, we evaluated both self-play and fixed-baseline scenarios. As shown in Fig. 6 (Bottom), adversarial training introduces

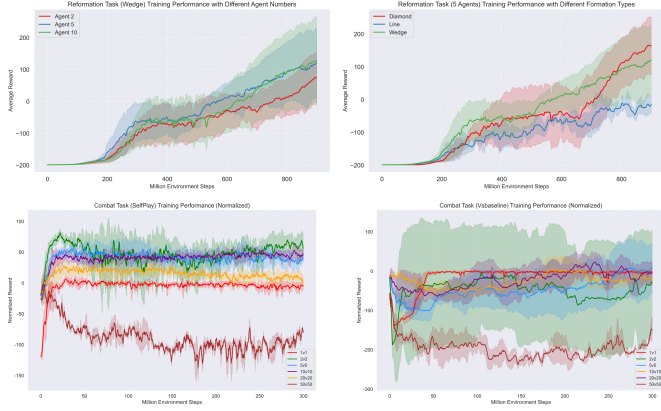
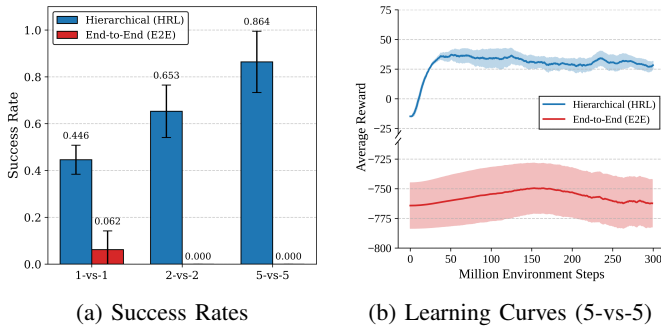


Fig. 6. **Learning Curves across Benchmark Tasks.** (Top) IPPO training returns for the decentralized formation keeping task across varying swarm sizes ($N = 2, 5, 10$) and geometric templates (Wedge, Diamond, Line). (Bottom) Hierarchical MAPPO returns in the adversarial pursuit-evasion benchmark under self-play and fixed-baseline settings, demonstrating successful convergence from 1-vs-1 up to massive 50-vs-50 engagements. Shaded regions denote the standard error across multiple random seeds.



(a) Success Rates

(b) Learning Curves (5-vs-5)

Fig. 7. **Impact of Hierarchical Policy Structure.** (Left) Comparison of success rates against a fixed-policy baseline. While HRL scales effectively with team size (reaching 86.4% in 5-vs-5), the E2E policy suffers a complete collapse in dense scenarios. (Right) Learning curves in the 5-vs-5 pursuit-evasion task, illustrating that the hierarchical decomposition is essential for stabilizing convergence in long-horizon, high-dynamic continuous control.

expected variance, yet the Centralized Training with Decentralized Execution (CTDE) paradigm successfully stabilizes the learning process.

A key observation from this benchmark is that hierarchical structure substantially improves optimization stability in high-dynamic pursuit-evasion settings. We conducted an ablation study comparing our two-tier Hierarchical RL (high-level target generation, low-level continuous tracking) against a naive End-to-End (E2E) continuous control policy. As depicted in Fig. 7, while the E2E policy entirely fails to converge in dense multi-agent settings (e.g., 5-vs-5) due to the compounding complexity of long-horizon credit assignment and non-linear aerodynamic bounds, the HRL architecture rapidly converges to a high success rate. These results indicate that Planax can support structured decision-making algorithms and expose optimization challenges that are difficult to observe in simplified point-mass benchmarks.

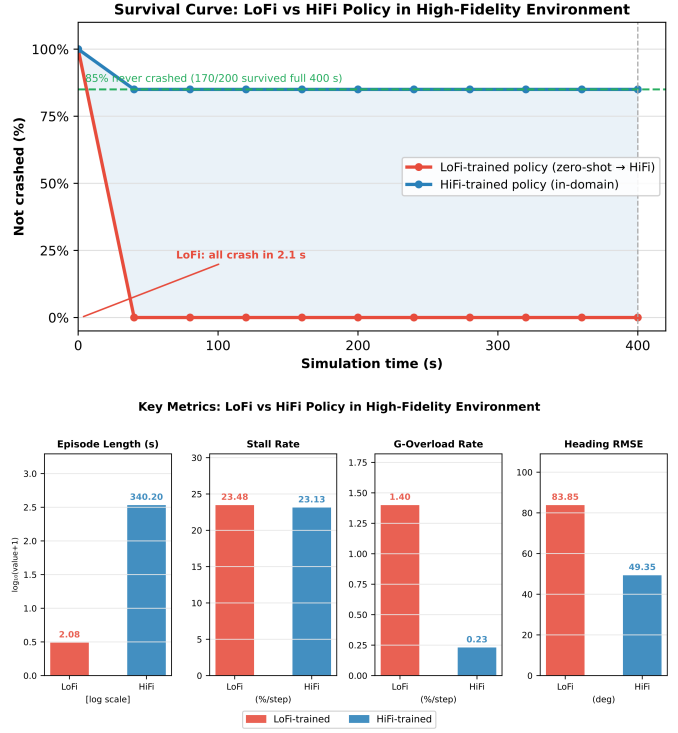


Fig. 8. **Sim-to-Real Proxy Validation.** (Top) Survival probability curves in the high-fidelity environment. The LoFi-trained policy suffers a 100% crash rate within average 2.1 seconds, while the Planax HiFi-trained policy maintains an 85% survival rate. (Bottom) Quantitative comparison of key physical metrics. The LoFi-trained policy fails primarily due to much shorter survival time, higher G-overload violation rate, and larger heading tracking error, while stall-rate statistics remain comparable under this test condition.

C. Fidelity Ablation: Impact of Aerodynamic Constraints

In the real world, trial-and-error for high-dynamic fixed-wing aerial robots is prohibitively expensive and physically catastrophic. To answer whether the massive computational effort required for 6-DOF high-fidelity aerodynamic modeling is truly necessary for MARL, we design a “Sim-to-Real Proxy” ablation study. We hypothesize that policies trained in simplified, low-fidelity (LoFi) kinematic environments exploit non-physical loopholes, rendering them undeployable. Therefore, the HiFi engine is used as a reference evaluation environment for testing whether policies trained under simplified dynamics remain valid when nonlinear aerodynamic constraints are restored.

1) *Ablation Design:* We utilize the Single-Agent Agile Control benchmark, requiring the F-16 to perform a continuous “S-maneuver” by tracking a target heading that alternates extreme angles every 10 seconds. Two policies are trained for 2,000 epochs: 1) LoFi Policy: Trained in a simplified dynamics environment, then deployed zero-shot into the HiFi environment. 2) HiFi Policy: Trained and evaluated in the same HiFi environment (domain matched). Both policies are evaluated across 200 parallel HiFi environments for a maximum horizon of 400 seconds.

2) *Quantitative Failure Analysis:* The evaluation yields a stark contrast, as summarized in Fig. 8. The HiFi Policy successfully adapts to the nonlinear coupling, maintaining an

85% survival rate through the 400-second episode with an average survival time of 340.2 seconds. It tracks the aggressive S-maneuver with a heading Root Mean Square Error (RMSE) of 49.35°.

Conversely, the LoFi Policy suffers a catastrophic 100% crash rate, surviving an average of merely 2.1 seconds before departing controlled flight. A deep analysis of the termination conditions reveals the root cause of this sim-to-real gap: the LoFi policy exhibits a G-overload violation rate of 1.40% per step (compared to 0.23% for the HiFi policy). Because the low-fidelity training environment did not penalize aggressive actuator inputs with realistic aerodynamic drag and inertia coupling, the LoFi policy learned dangerous, "shortcut" control behaviors. When subjected to the strict energy-maneuverability constraints of the HiFi environment, these commands immediately induce structural G-limit violations or accelerated stalls.

These results suggest that simplified training dynamics can induce unsafe shortcut behaviors when policies are evaluated under nonlinear aerodynamic constraints. Therefore, high-fidelity aerodynamic modeling is important not only for visual realism, but also for exposing safety-relevant failure modes during policy evaluation. This controlled ablation supports the use of Planax as a benchmark for studying physically grounded fixed-wing aerial reinforcement learning.

V. CONCLUSION

This letter presented Planax, a JAX/XLA-accelerated benchmark framework for large-scale fixed-wing aerial reinforcement learning. By integrating Tensor-LUT aerodynamics and nonlinear 6-DOF dynamics into GPU-resident compiled roll-outs, Planax provides a scalable testbed for studying single-agent agile tracking, cooperative formation, and adversarial pursuit-evasion under flight-envelope constraints. Experiments demonstrate high rollout throughput, favorable memory scaling, and stable training of representative PPO-family baselines. Fidelity ablations further show that simplified dynamics can induce unsafe shortcut behaviors when evaluated under high-fidelity aerodynamic constraints. Future work will extend the benchmark toward broader airframes, richer atmospheric effects, and more diverse robotic aerial tasks.

REFERENCES

- [1] J. S. Berndt, "Jsbsim: An open source flight dynamics model in C++," in *AIAA Modeling and Simulation Technologies Conference*, 2004, p. 4923.
- [2] Laminar Research, "X-plane flight simulator," <https://www.x-plane.com/>, 2025.
- [3] J. Panerati, H. Zheng, S. Zhou, J. Dong, Z. Grey, and A. Schoellig, "Learning to fly—a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control," *IEEE Robotics and Automation Letters*, vol. 7, no. 1, pp. 1228–1235, 2021.
- [4] Y. Song, S. Naji, E. Kaufmann, A. Loquercio, and D. Scaramuzza, "Flightmare: A flexible quadrotor simulator," in *Proceedings of the 4th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 155, 2021, pp. 1147–1157.
- [5] B. Xu, F. Gao, C. Yu, R. Zhang, Y. Wu, and Y. Wang, "OmniDrones: An efficient and flexible platform for reinforcement learning in drone control," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2838–2844, 2024.
- [6] V. Makoviychuk *et al.*, "Isaac Gym: High performance GPU-based physics simulation for robot learning," 2021.
- [7] C. Xue *et al.*, "Neuralplane: An efficiently parallelizable platform for fixed-wing aircraft," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [8] J. Bradbury *et al.*, "JAX: Composable transformations of Python+NumPy programs," <https://github.com/google/jax>, 2018.
- [9] D. Freeman *et al.*, "Brax: A differentiable physics engine for large scale rigid body simulation," 2021.
- [10] A. Rutherford *et al.*, "JaxMARL: A suite of multi-agent reinforcement learning environments and algorithms in JAX," 2023.
- [11] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "RLBench: The robot learning benchmark and learning environment," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [12] Z. Yuan, A. W. Hall, S. Zhou, L. Brunke, M. Greeff, J. Panerati, and A. P. Schoellig, "safe-control-gym: A unified benchmark suite for safe learning-based control and reinforcement learning in robotics," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11 142–11 149, 2022.
- [13] R. Lowe *et al.*, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [14] M. Samvelyan *et al.*, "The StarCraft multi-agent challenge," in *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2019, pp. 2186–2188.
- [15] C. Richter *et al.*, "QPlane: An open-source reinforcement learning toolkit for autonomous fixed wing aircraft simulation," in *Proceedings of the ACM Symposium on Applied Computing (SAC)*, 2021, pp. 1065–1072.
- [16] A. Pope *et al.*, "Hierarchical reinforcement learning for air combat," *IEEE Transactions on Artificial Intelligence*, vol. 3, no. 6, pp. 878–889, 2022.
- [17] Y. Jiang *et al.*, "Learning agile fighter maneuvers using hierarchical multi-agent reinforcement learning," *IEEE Transactions on Neural Networks and Learning Systems*, 2024, early Access.
- [18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.
- [19] C. Yu *et al.*, "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 611–24 624.